

MSE 312: MECHATRONICS DESIGN II**Integration Lab 2: Reading Quadrature Incremental Encoders****OBJECTIVE**

- To become familiar with how to measure angular displacement using a quadrature encoder

Required Components/Material:

- Matlab/Simulink 2021b
- MEGA 2560 R3 Board ATmega2560 ATMEGA16U2 + USB Cable for Arduino
- DC Servomotor with encoder (e.g., GM8224S009)

References:

- Arduino Support from MATLAB/Simulink (www.mathworks.com)
- Rensselaer Arduino Support Package Library (RASPLib)
<https://www.mathworks.com/matlabcentral/fileexchange/62702-reysselaer-arduino-support-package-library-rasplib>

PROCEDURE**ACTIVITY 2.1: Rensselaer Arduino Support Package - RASPLib Installation**

Rensselaer Polytechnic Institute has developed blocks that may be used for motor control including PWM waveform generation and reading rotary quadrature encoders. To use these blocks, the following steps should be followed. A folder named “RASPLib” and a script named “startup.m” will be provided to you on canvas.sfu.ca:

1. Copy and paste “startup.m” and “RASPLib” to your Matlab working directory, for example a directory on DESKTOP where your Simulink files are.
2. Run Matlab as administrator (right click on icon and choose “run as administrator”), then open and Run “startup.m” to add the directories to Matlab path. This will cause the Rensselaer Arduino Support Package entry to be added to the Simulink library browser as illustrated in the figure below. This process may take a while for the first time.
3. Open the demo file for your particular hardware. The examples for the Arduino hardware is under the M1V4 Mega option.
4. After opening the Demo, save it to your home directory with “save-as”. From now on, you can create Simulink diagrams in any location by dragging the blocks from the library.

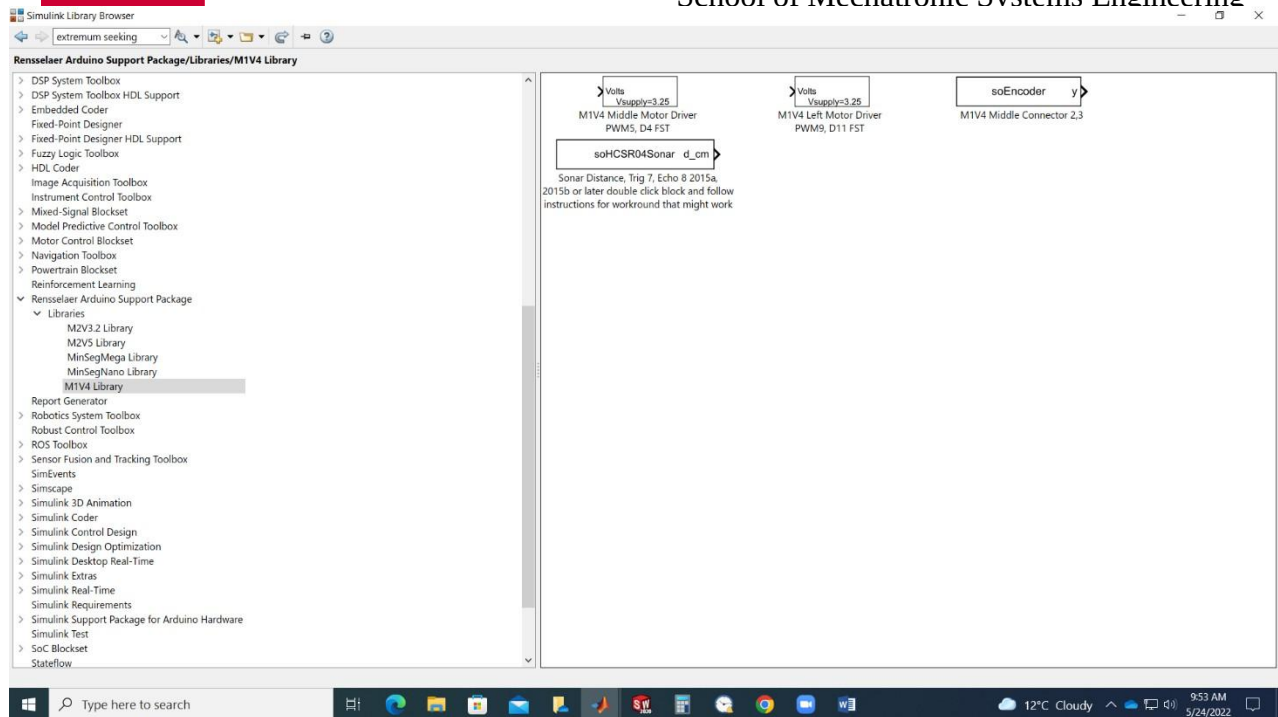


Figure 2.1: Rensselaer Arduino Support Package Simulink blocks.

ACTIVITY 2.1: Reading the Incremental Encoder

For precision motor control, a quadrature incremental encoder may be used to measure the motor shaft's angular displacement. In this experiment, we assume that the encoder generates 500 counts per rotation (CPR). Figure 3.1 below shows 5 wires connected to the encoder and the internal blocks of the associated circuitry. The motor is equipped with a gearbox reduction ratio of 6.3:1.

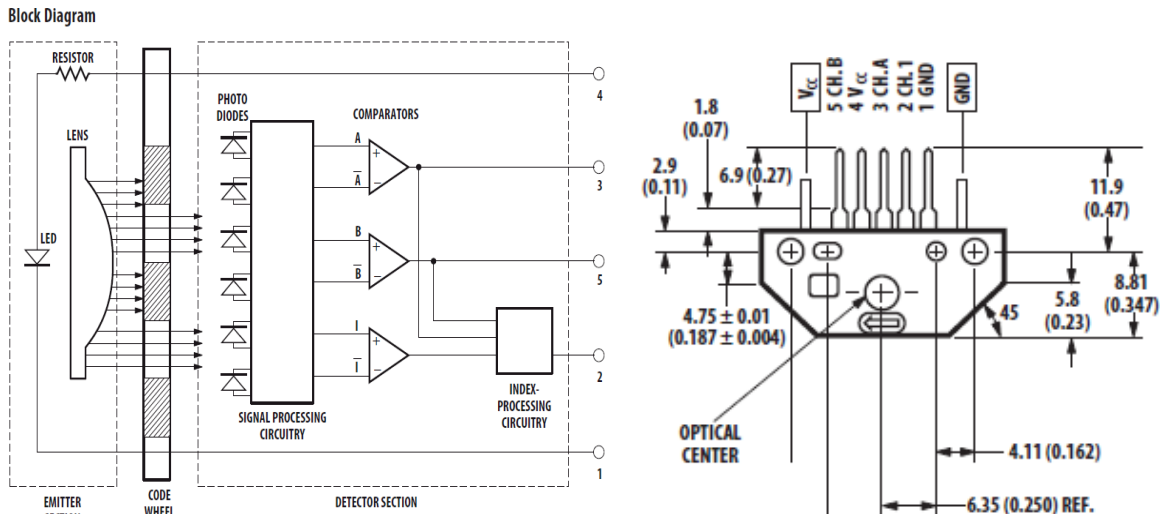


Figure 2.2: Incremental encoder blocks and color coded wirings:
Blue (Ch B), Red (Vcc), Yellow (Ch A), Green (Ch I), Black (GND)

Example 2.1: Read rotary encoder position in Matlab

1. Create an Arduino object with the following command

```
>> a = arduino('COM3','Mega2560','Libraries',{ 'I2C', 'RotaryEncoder', 'SPI', 'Servo',  
'ShiftRegister'})
```

You should get a response like the following

```
a = arduino with properties:
```

```
    Port: 'COM3'
```

```
   Board: 'Mega2560'
```

```
AvailablePins: {'D2-D53', 'A0-A15'}
```

```
Libraries: {'I2C', 'RotaryEncoder', 'SPI', 'Servo', 'ShiftRegister'}
```

2. Specify two digital pins in interrupt mode to be used in “rotaryEncoder” command to read from the two encoder channels. By default, Interrupt pins for board Mega2560 are 'D2', 'D3', 'D18', 'D19', 'D20', 'D21'. You can inspect each pin’s mode using the following command:

```
configurePin(a, 'D3');
```

3. You should get a response like the following

```
ans =  
  
    'unset'
```

4. To change the pin into Interrupt, use the following code

```
configurePin(a, 'D2', 'Interrupt');  
configurePin(a, 'D3', 'Interrupt');
```

5. Create a “rotaryEncoder” Object using the following command

```
>> encoder = rotaryEncoder(a,'D2','D3',500)
```

You should get a response like the following

```
encoder =
```

```
    rotaryEncoder with properties:
```

```
    ChannelA: 'D2'
```

```
    ChannelB: 'D3'
```

```
 PulsesPerRevolution: 500
```

Use the following command to reset the encoder count before you start reading

```
resetCount(encoder);
```

Connect Ch B, Ch A, Vcc, and GND of the encoder to the corresponding pins on the microcontroller board (D2, D3, 5V, GND). Then use the following commands to read the encoder count continuously and change the motor knob position to see different readings

```
while(1)
count = readCount(encoder)
pause(1);
end
```

This will print out the current count value of the encoder with X4 decoding. The counter increments or decrements similarly on each edge of channels A and B for X4 encoding. Whether the counter increments, or decrements, depends on which channel leads the other.

Each cycle results in four increments or decrements. Once you have set the encoding type and counted the pulses, you can use the following formula to obtain the angular displacement

$$Position(^{\circ}) = \frac{Edge\ Count}{Q \cdot N \cdot G} \times 360$$

Where N = 500 is the number of pulses generated by the encoder per shaft revolution, Q is encoding type (4 for quadrature), and G= 6.3 is the motor gear ratio.

Make sure to use “clear all” in Matlab before doing the next experiment.

Example 2.2: Read Rotary Encoder Knob position in Simulink

1. To read from the Encoder channels using the Arduino, Drag and drop the “SoEncoder” block from *Rensselaer Arduino Support Package* > > *open library* > > *M1V4 library* as the following image shows.

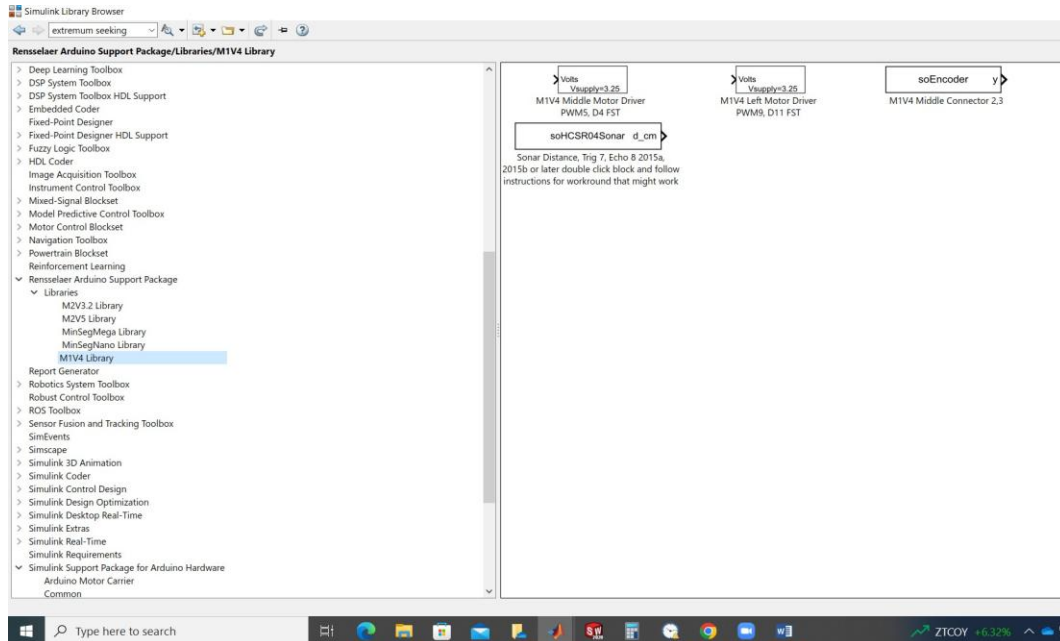


Figure 2.3: Encoder interface block.

2. In the encoder block parameters, choose the encoder number and corresponding pins that you will connect to channels A and B of the motor encoder.

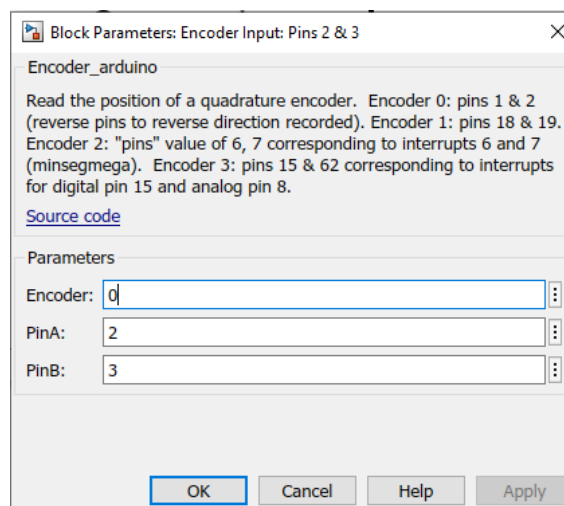


Figure 2.4: Encoder input block parameters.

3. Use a conversion block to convert the data type of the counted value to *double*. Drag and drop a “MATLAB function” block from *Simulink>>user defined functions* to implement the formula in example 2.1 to convert count to angular position in radians.
4. Connect a display and a scope to the output signal to see the position.
5. Use the **Model configuration instructions** Appendix to configure the model to be deployed on the Arduino hardware and run the simulation. Change the encoder shaft position manually and compare it with the results.

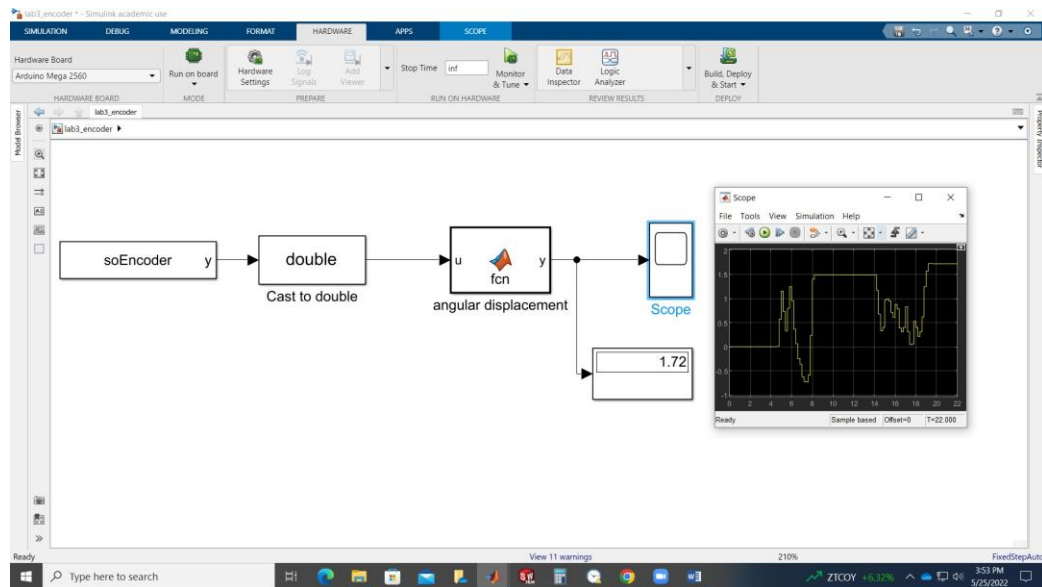


Figure 2.5: Reading a rotary encoder.

Note: If you receive an error, double check and make sure that you have cleared parameters in script.

Model configuration instructions
(detailed images in example 1.3 (Integration Lab 1))

1. From the dropdown menu by the gear icon on top of the Simulink model, choose the model configuration parameters option.
2. From the configuration parameters window that opens, choose hardware implementation on the left hand side menu and select the Arduino Mega 2560 as the hardware board
3. After choosing the Mega 2560 as your Hardware board, the Target hardware resources option appears at the bottom of the window. From Groups, select Host-board connection and choose the set host COM port as manually and insert the COM port number your Arduino is connected to (e.g., COM 3). Click OK to apply the changes.
4. In the the Solver tab, for the Start time field, enter 0. In the Stop time field, enter the amount of time you want your model to run. For example, enter inf if you want to run the program indefinitely. From the Type list, choose Fixed-step. From the Solver list, choose a solver. For example, ode5 (Dormand-Prince). In the Fixed step size field, enter a sample time. For example, enter 0.01 seconds for a sample rate of 100 samples/second . This sample time should be the same as the sample time in your blocks. It is recommended to keep the sample time constant in all blocks.
5. In the simulink model window, run the simulation from HARDWARE tab.